

Intelli - Music

Clustering e Ricerca

Gruppo di Lavoro:

Colella Stefano, [MAT. 775267], S.Colella27

Link GitHub: [Intelli-Music](#)

‘<https://github.com/Step0491/Intelli-Music.git>’

A.A. 2024-2025

Indice (*Linkato*)

Introduzione	3
Dataset	4
Apprendimento non supervisionato	10
Apprendimento supervisionato	14
Ragionamento probabilistico e Bayesian Network.....	30
Valori continui.....	31
Valori discreti.....	34
Ragionamento logico e Prolog	35
Sviluppi futuri	37

Introduzione

Intelli-Music è un sistema che si pone l'obiettivo di aiutare l'utente nel comprendere al meglio quali potrebbero essere le migliori playlist da creare/utilizzare a partire da un insieme di canzoni. (Nel progetto si è utilizzato un dataset contenente canzoni e varie features che hanno permesso il clustering).

Il sistema potrà inserire nuove canzoni all'interno delle playlist create tramite clustering.

Requisiti funzionali

Il progetto è stato realizzato in Python in quanto linguaggio di alto livello che semplifica la programmazione con svariate librerie che permettono di trattare i dati in modo facile e intuitivo. Versione Python: 3.12

IDE utilizzato: Visual Studio Code

Librerie utilizzate:

- **matplotlib**: visualizzazione dei grafici (curve di apprendimento, grafici a barre e a torta)
- **networkx**: visualizzazione di grafi (generazione della rete bayesiana)
- **numpy**: libreria per gestire arrayx
- **pandas**: importazione e manipolazione dei dataset .csv
- **pgmpy**: creazione della rete bayesiana
- **scikit_learn**: libreria utilizzata per apprendimento automatico
- **pyswip**: utilizzo di Prolog all'interno dell'ambiente python
- **imblearn**: libreria utilizzata per oversampling tramite SMOTE

Installazione e avvio

Aprire il progetto con l'IDE preferito. Avviare il programma partendo dal file **main.py**.

Eseguire **removeUnnecessaryFiles.py** per eliminare tutti i file non necessari e poter svolgere un'esecuzione pulita dell'app se necessario.

Dataset

Il dataset è stato scaricato dalla piattaforma Kaggle. Visti gli scopi principali del progetto, ovvero produrre un sistema funzionante su dataset generici, mi è sembrato superfluo aggiungere le Api di Spotify data la funzionalità con qualsiasi dataset di streaming musicale che rispetti le features implementate; trattasi delle stesse che Spotify utilizza per descrivere le sue canzoni nella documentazione:

- **Danceability** : La danceability descrive quanto una traccia sia adatta per ballare in base a una combinazione di elementi musicali, tra cui tempo, stabilità del ritmo, forza del ritmo e regolarità generale
- **Energy** : L'energia è una misura da 0,0 a 1,0 e rappresenta una misura percettiva di intensità e attività. In genere, le tracce energiche sembrano veloci, forti e rumorose. Le caratteristiche percettive che contribuiscono a questo attributo includono la gamma dinamica, il volume percepito, il timbro, la velocità di insorgenza e l'entropia generale

- **Key** : La tonalità in cui si trova la traccia. I numeri interi vengono mappati sulle altezze utilizzando la [notazione standard Pitch Class](#). Se non è stata rilevata alcuna chiave, il valore è -1. Intervallo: -1 - 11
- **Loudness** : Il volume complessivo di una traccia in decibel (dB). I valori del volume vengono calcolati in media sull'intera traccia e sono utili per confrontare il volume relativo delle tracce. Il volume è la qualità di un suono che è il principale correlato psicologico della forza fisica (ampiezza). I valori variano tipicamente tra -60 e 0 db.
- **Speechiness** : La *speechiness* rileva la presenza di parole pronunciate in una traccia. Più la registrazione è esclusivamente vocale (ad esempio talk show, audiolibro, poesia), più il valore dell'attributo si avvicina a 1.0. Valori superiori a 0,66 descrivono brani probabilmente costituiti interamente da parole pronunciate. I valori compresi tra 0,33 e 0,66 descrivono tracce che possono contenere sia musica che parlato, in sezioni o sovrapposti, inclusi casi come la musica rap. I valori inferiori a 0,33 rappresentano molto probabilmente musica e altri brani non simili al parlato.
- **Acousticness** : Una misura di confidenza da 0,0 a 1,0 che indica se la traccia è acustica. 1.0 rappresenta un'elevata certezza che la traccia sia acustica.
- **Instrumentalness** : Prevede se una traccia non contiene parti vocali. I suoni "Ooh" e "aah" sono trattati come strumentali in questo contesto. Più il valore di strumentalità è vicino a 1.0, maggiore è la probabilità che la traccia non contenga contenuto vocale. I valori superiori a 0,5 intendono rappresentare brani strumentali, ma la sicurezza aumenta quando il valore si avvicina a 1,0.

- **Liveness** : Rileva la presenza di un pubblico nella registrazione.
Valori di vivacità più elevati rappresentano una maggiore probabilità che la traccia sia stata eseguita dal vivo. Un valore superiore a 0,8 fornisce una forte probabilità che la traccia sia live.
- **Valence** : Una misura da 0,0 a 1,0 che descrive la positività musicale trasmessa da una traccia. Le tracce con alta valenza suonano più positive (ad esempio felice, allegro, euforico), mentre le tracce con bassa valenza suonano più negative (ad esempio triste, depresso, arrabbiato).
- **Tempo** : Il tempo complessivo stimato di una traccia in battiti al minuto (BPM). Nella terminologia musicale, il tempo è la velocità o il ritmo di un dato brano e deriva direttamente dalla durata media del beat.
- **Mode** : Indica se una traccia è in **scala maggiore (1)** o **minore (0)**; è determinato in base alla tonalità della canzone e aiuta nelle **raccomandazioni musicali**, poiché le canzoni in modo maggiore tendono a suonare più felici e luminose, mentre quelle in modo minore sono spesso percepite come più tristi o cupe.
Viene utilizzato negli **algoritmi di creazione delle playlist personalizzate** (es. "Discover Weekly").
- **Duration_ms** : Indica la durata totale della traccia in **millisecondi** (1 secondo = 1000 ms). Un valore tipico è **tra 180.000 e 300.000 ms** (3-5 minuti) ma nel dataset l'intervallo risulta più ampio.

Preprocessing del dataset

Il preprocessing del dataset è stato utilizzato per la creazione del dataset realmente utilizzato successivamente, memorizzato nel file **cleanDataset.csv**. Il preprocessing si è dimostrato fondamentale per i seguenti motivi:

- 1) Rimozione di features come: Track_name, Track_id, lyrics, Genre e Subgenre...
- 2) Una colonna aggiuntiva (**clusterIndex**) è aggiunta e utilizzata solo per visualizzare come vengono generate le playlist attraverso clustering. Poiché l'obiettivo è organizzare delle playlist secondo le sole feature delle canzoni.
- 3) L'algoritmo di clustering utilizzato, ovvero il *KMeans*, può essere influenzato da scale diverse di valori numerici. Poiché l'algoritmo utilizza la distanza euclidea tra i punti dati per assegnarli a cluster, le caratteristiche con scale molto diverse possono avere un impatto significativo sui risultati
- 4) Senza preprocessing l'addestramento della rete Bayesiana portava a un eccessivo uso di memoria per il computer, facendo interrompere il programma.
Questo problema è stato risolto eseguendo la discretizzazione dei valori.

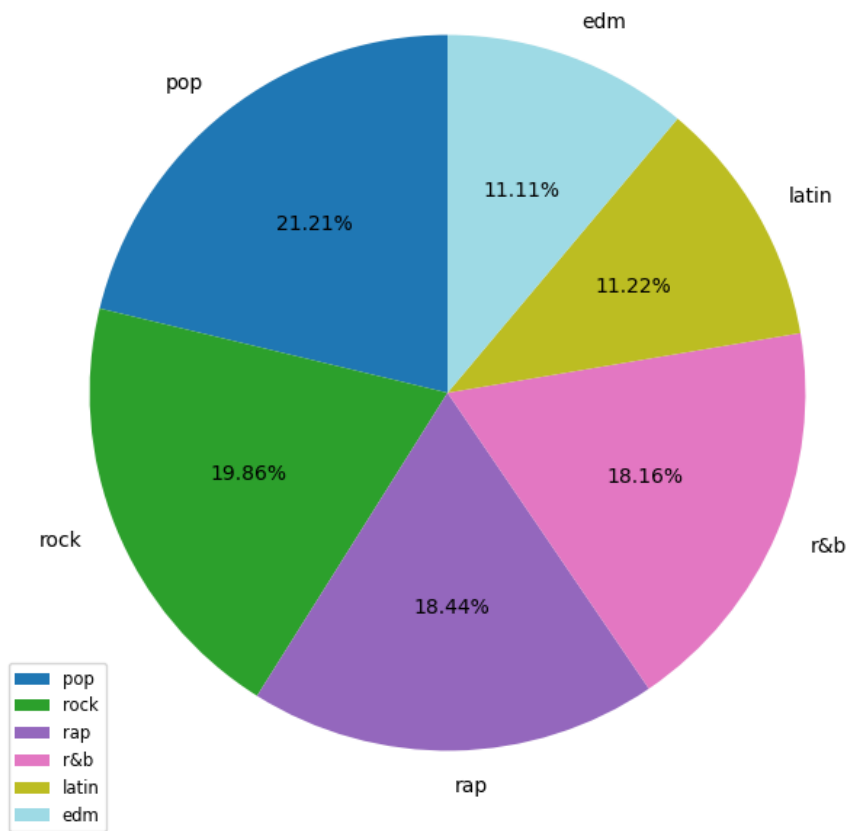
E' stato utilizzato un oggetto di tipo *MinMaxScaler* per **normalizzare** i valori delle feature nel dataset portandoli tutti quanti in un intervallo compreso tra 0 e 1. In particolare, la normalizzazione Min-Max si ottiene sottraendo al valore di una determinata feature il valore minimo di tale feature e dividendo per la differenza tra il massimo e il minimo della determinata feature.

Per la successiva **discretizzazione** è stato usato un oggetto di tipo *KBinsDiscretizer*; una classe di Scikit-Learn che permette di trasformare variabili numeriche continue in variabili discrete/categoriali, suddividendo i valori in bin (intervalli). Utile per:

- Discretizzare variabili numeriche per modelli che funzionano meglio con categorie (es. alberi decisionali, reti bayesiane).
- Ridurre la granularità dei dati semplificando l'analisi.
- Migliorare le prestazioni dei modelli che non lavorano bene con valori continui.

Il dataset inizialmente si mostrava nel seguente modo:

Distribuzione Generi Playlist



Apprendimento non supervisionato

L'apprendimento non supervisionato è una branca dell'apprendimento automatico in cui l'agente viene addestrato su un insieme di dati senza etichette. Esistono due principali tipi di apprendimento non supervisionato:

- Clustering : L'obiettivo è raggruppare gli elementi del dataset in base a delle somiglianze
- Riduzione della dimensionalità : L'obiettivo è ridurre il numero di feature utilizzate mantenendo però le informazioni più significative.

Un esempio è la *PCA (Principal Component Analysis)*

Nel mio caso l'apprendimento non supervisionato è stato impiegato al fine di effettuare clustering per raggruppare le canzoni in gruppi che formeranno le mie nuove playlist. Esistono due tipologie di apprendimento di clustering: **hard clustering** e **soft clustering**.

Hard clustering: associa ogni esempio a un cluster specifico

Soft clustering: il secondo associa a ogni esempio la probabilità di appartenenza a ogni cluster.

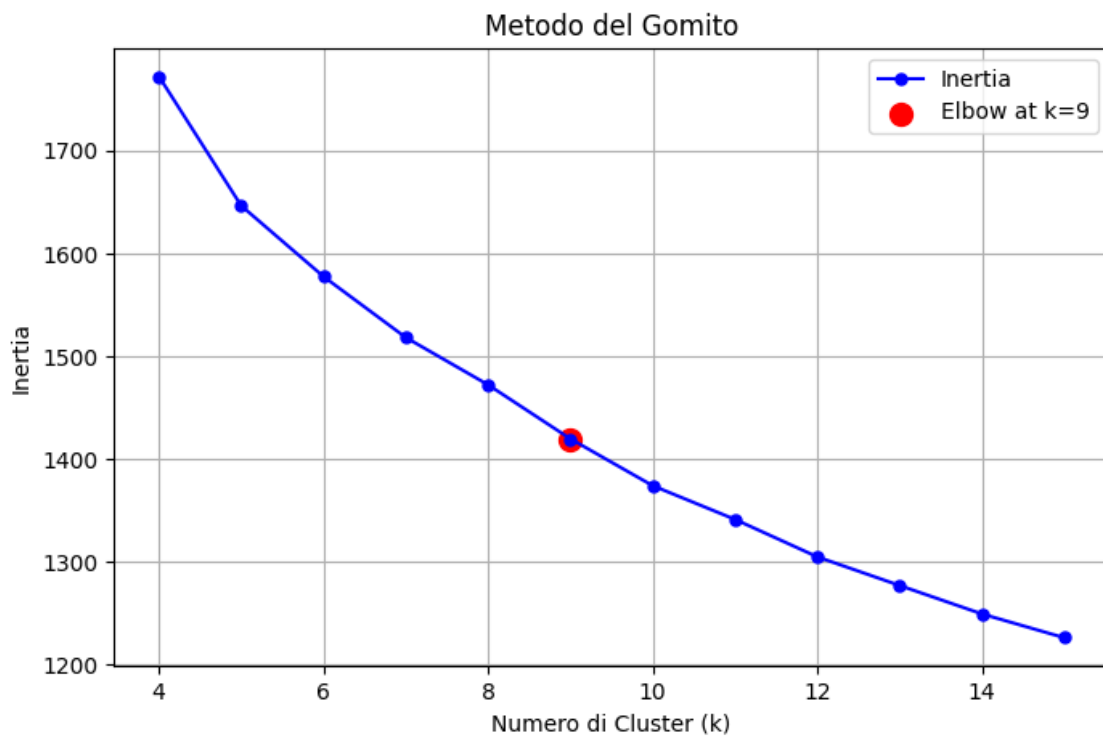
Per la realizzazione di questa parte è stata usata la libreria *sklearn* per il clustering, la libreria *matplotlib* per la visualizzazione di grafici e la libreria *kneed* per individuare il gomito.

Ho deciso di implementare l'algoritmo di hard clustering *KMeans* pur sapendo che uno dei suoi problemi principali è appunto trovare il numero ideale di cluster da fornire in input all'algoritmo. Per risolvere questo problema ho seguito una strategia nota come "curva del gomito".

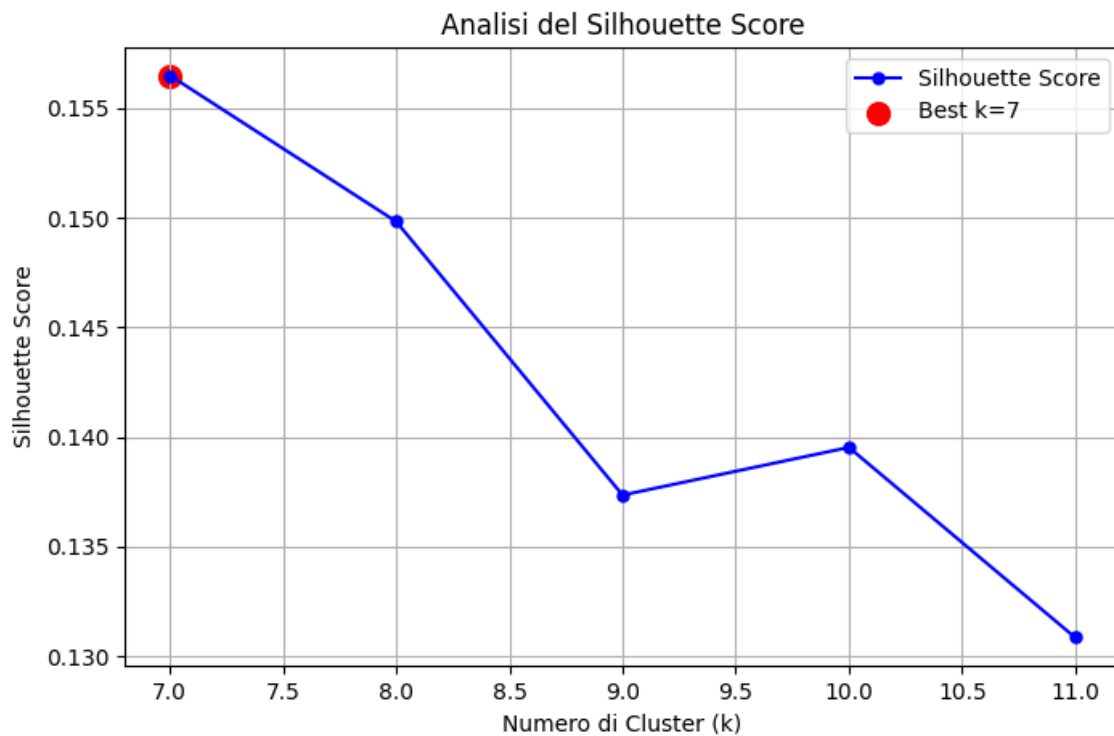
- La **curva del gomito** mostra la variazione dell'*inertia* al variare del numero di cluster; l'*inertia* rappresenta la somma delle distanze quadrate tra ogni punto dati e il centro del cluster assegnato. L'algoritmo prevede dunque di eseguire il *KMeans* per diversi valori di numero di cluster, calcolare l'*inertia* per ognuno di essi, plottare su un grafico la curva e poi identificare il gomito, ovvero il punto in cui la diminuzione dell'*inertia* diventa meno significativa. Poi viene affinato tale valore K attraverso il **silhouette score** in range ± 2 .
- Il clustering viene eseguito variando il numero di cluster da **4 a maxK=15** evitando il rischio di avere cluster troppo piccoli o non significativi.

Per ogni k, viene calcolata l'*inertia* sfruttando la **parallelizzazione** per velocizzare i calcoli.

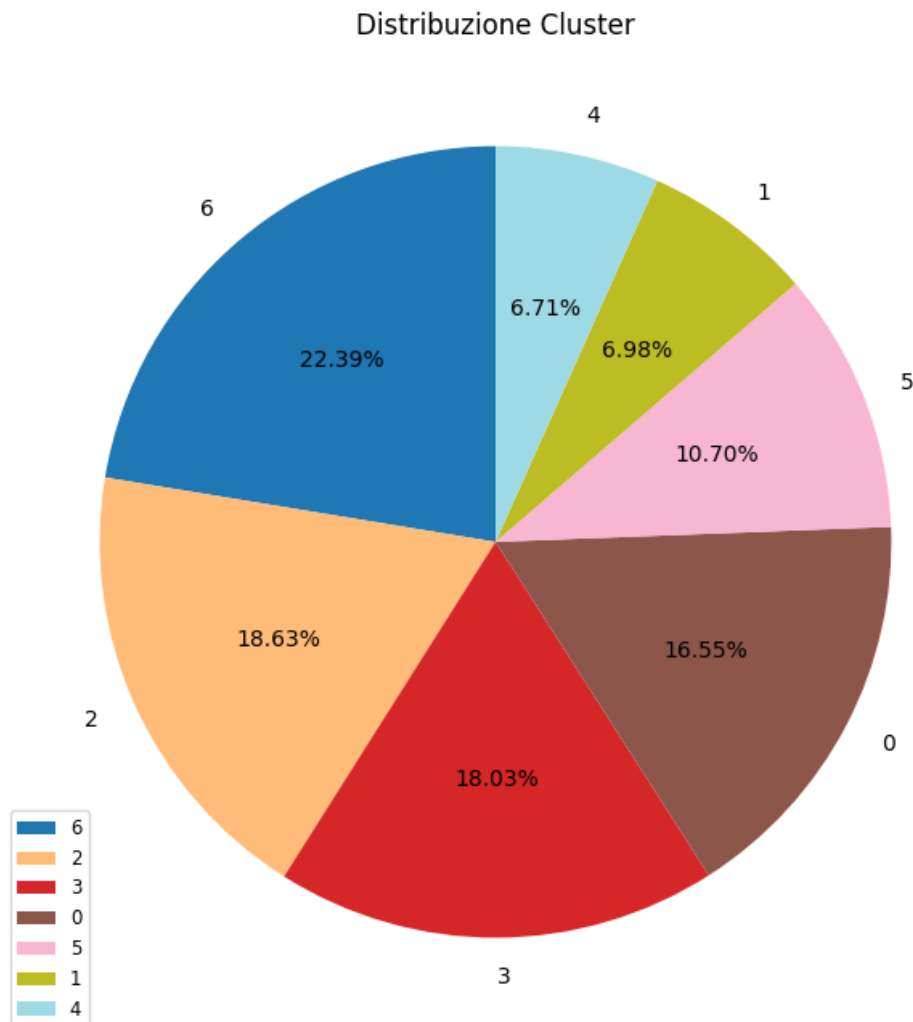
Per ogni valore di k l'algoritmo viene eseguito **10 volte** (n_init=10) per ottenere risultati più stabili. L'inizializzazione dei centroidi utilizza il metodo **k-means++**, che garantisce un'avvio migliore rispetto a una scelta completamente casuale, riducendo il rischio di convergenza a minimi locali.



Nel mio caso possiamo vedere come il numero ottimale di cluster è $k = 9$, affinato con silhouette score ± 2 ottenendo un $k=7$ finale.



Dunque è stato eseguito l'algoritmo *KMeans* con 7 cluster. Il risultato del clustering è il seguente:



Ogni cluster viene identificato da un indice (`clusterIndex`). Come possiamo vedere la distribuzione delle canzoni nei vari cluster non è perfettamente bilanciata. Ciò potrebbe influire su alcuni algoritmi di apprendimento supervisionato. Nel seguito verrà trattata la questione relativa allo sbilanciamento delle classi e alla sua possibile soluzione.

Apprendimento supervisionato

L'apprendimento supervisionato è una branca dell'apprendimento automatico in cui l'agente viene addestrato su un insieme di dati con etichette. Si divide in

- **Classificazione:** Le etichette rappresentano un dominio di valori discreto (quindi un insieme finito di valori) che la feature di output può assumere. Un caso particolare è la classificazione booleana, in cui le classi sono **true** e **false**.
- **Regressione:** L'etichetta può essere un qualsiasi valore numerico, quindi il dominio della feature di output è continuo.

L'obiettivo principale è far sì che l'algoritmo *apprenda* una relazione tra gli input e gli output in modo che, una volta addestrato, possa fare previsioni accurate su nuovi dati mai visti prima.

Ci sono diverse fasi da affrontare:

- 1) Scelta degli iper-parametri
- 2) Fase di addestramento
- 3) Fase di test
- 4) Valutazione delle prestazioni

Per questa parte di progetto sono state utilizzate le librerie: *matplotlib* e *numpy* per la visualizzazione di grafici, *sklearn* per l'apprendimento supervisionato, *pandas* e *imblearn.over_sampling* per l'oversampling del dataset.

Nel mio caso l'apprendimento supervisionato è stato utilizzato con scopo di classificazione, dove la classi sono gli indici dei cluster prima ottenuti mediante apprendimento non supervisionato.

Per questo progetto ho deciso di utilizzare tre modelli di apprendimento automatico:

- **DecisionTree** : Classificatore strutturato ad albero in cui le foglie rappresentano le classi di appartenenza (o le probabilità di appartenenza a tali classi) mentre la radice e i nodi interni rappresentano delle condizioni sulle feature di input. A seconda se tali condizioni sono rispettate o meno, verrà seguito un percorso piuttosto che un altro e alla fine si arriverà alla classe di appartenenza
- **RandomForest** : Classificatore che si ottiene generando più DecisionTree. Il valore in output si ottiene mediando sulle predizioni di ogni albero appartenente alla foresta (tecnica di bagging)
- **Logistic Regression**: A differenza della classica *regressione lineare*, che è utilizzata per predire valori continui (regressione), la **logistic regression** viene utilizzata per calcolare le probabilità di appartenenza di un esempio a ogni classe.

Per ogni classe, si apprende una funzione lineare i cui pesi vengono trovati tramite discesa di gradiente. Il risultato viene trasformato dalla funzione **softmax** in una distribuzione di probabilità tra le classi. Il vantaggio è dunque poter apprendere $n-1$ funzioni (date n le classi) e ottenere la probabilità dell'ultima classe come $1 - \sum$ delle altre probabilità apprese. Il risultato è la categoria con la probabilità maggiore

Fase 1) Scelta degli iper-parametri

Gli iper-parametri sono i parametri di un modello di apprendimento automatico, i quali non vengono appresi durante la fase di addestramento come i normali parametri del modello (es. i pesi di una funzione lineare) ma devono essere necessariamente fissati prima che il modello possa cominciare l'addestramento. La loro scelta influisce sulle prestazioni e sulla complessità del modello. Uno dei compiti più complessi è proprio la scelta degli iper-parametri per i vari modelli.

Per la scelta degli iper-parametri ho utilizzato una tecnica di ***K-Fold Cross Validation (CV)***.

Nella *K-Fold CV* il dataset viene diviso in k fold (insiemi disgiunti) e il modello viene addestrato k volte. Per ogni iterazione 1 fold viene usato per il testing mentre gli altri $k-1$ fold vengono utilizzati per il training. In questo modo è possibile testare e addestrare il modello su dati diversi per comprendere “la bontà” del modello.

La strategia che ho deciso di applicare per ricercare gli iper-parametri dei miei modelli è la *GridSearch* con *Cross Validation*. In questo approccio vengono definite le griglie dei valori possibili per gli iper-parametri e si esplorano tutte le combinazioni possibili alla ricerca della miglior combinazione possibile.

IPER-PARAMETRI RICERCATI e UTILIZZATI

DecisionTree

- **Criterion** : Misura la qualità dello split effettuato sui nodi. Può assumere i seguenti valori:
 - gini : Misura la “purezza” della divisione dei dati, più precisamente quanto spesso un elemento viene classificato in modo sbagliato
 - entropy : Misura la quantità di disordine nei dati. Minimizzare l’entropia significa massimizzare l’informazione guadagnata durante
- **Max_depth** : Indica l’altezza massima dell’albero
- **Min_samples_split** : Il numero minimo di esempi necessari affinché possa essere inserito un criterio di split. Se il numero è minore viene innestata una foglia
- **min_samples_leaf** : Il numero minimo di esempi per poter creare una foglia

RandomForest

Per quanto riguarda la creazione dei singoli alberi ho utilizzato gli stessi criteri del DecisionTree. Per quanto riguarda i criteri relativi alla foresta abbiamo:

- **n_estimators**: il numero di alberi nella foresta

LogisticRegression (Multinomial)

- **Penalty** : Stabilisce la penalizzazione applicata al modello, per ridurre la complessità e cercare di evitare *overfitting*
 - l1 : Viene applicata la norma 1 come termine di penalizzazione (ovvero la somma dei valori assoluti dei pesi). Viene anche detta regolarizzazione lasso. Tende a ridurre a 0 i pesi delle feature non importanti. In questo caso non è stata utilizzata in quanto la *lbfgs* non la supporta
 - l2 : Viene applicata la norma 2 come termine di penalizzazione (ovvero la somma dei quadrati dei pesi). Favorisce pesi più bilanciati, utile per prevenire *overfitting*.
- **C** : Specifica quanto “forte” è la regolarizzazione. Più il valore di C è basso e più forte è la regolarizzazione. Valori di C piccoli prevengono *overfitting* ma rendono il modello più rigido. Valori alti rendono il modello più flessibile ma sono soggetti a *overfitting*
- **Max_iter** : Indica il numero di iterazioni utilizzate
- **Solver** : Indica l'algoritmo da utilizzare per apprendere i pesi
 - lbfgs : mantiene solo alcune informazioni delle derivate parziali, utilizzando dunque poca memoria. È adatto per problemi con numero di feature elevato, soprattutto quando la memoria a disposizione non è molta
 - liblinear : Basato su algoritmi di programmazione lineare

Nel mio caso ogni modello è stato addestrato con un valore di *5 fold*
ovvero *5 iterazioni*

I parametri che sono stati restituiti sono:

Modello	Iper Parametro	Valore
DecisionTree	criterion	entropy
DecisionTree	Max_depth	10
DecisionTree	Min_samples_leaf	5
DecisionTree	Min_samples_split	5
RandomForest	n_estimators	50
RandomForest	Max_depth	10
RandomForest	Min_samples_leaf	1
RandomForest	Min_samples_split	2
LogisticRegression	C	10
LogisticRegression	Max_iteration	1000
LogisticRegression	Penalty	l2
LogisticRegression	solver	lbfgs

Fase 2) Addestramento e test

Per la fase di addestramento e di test i modelli sono stati addestrati utilizzando una *K-Fold Cross Validation* con $K = 5$.

Questa tecnica aiuta a ridurre la **varianza** nei risultati e fornisce una stima più affidabile della performance del modello.

Per ogni modello viene calcolata:

- **Accuratezza media** su tutte le fold.
- **Deviazione standard** per verificare la stabilità delle performance.

Il dataset viene suddiviso in training set e test set con una proporzione dell'80%-20%; successivamente valutiamo i modelli.

Metriche calcolate:

- **Accuracy**: Percentuale di campioni classificati correttamente.
- **Precision**: Percentuale di veri positivi tra quelli predetti come positivi.
- **Recall**: Percentuale di veri positivi identificati sul totale dei positivi reali.
- **F1-Score**: Media armonica tra Precision e Recall.

Per ogni modello, viene generata una **learning curve** che mostra l'errore di training e test in funzione della dimensione del dataset.

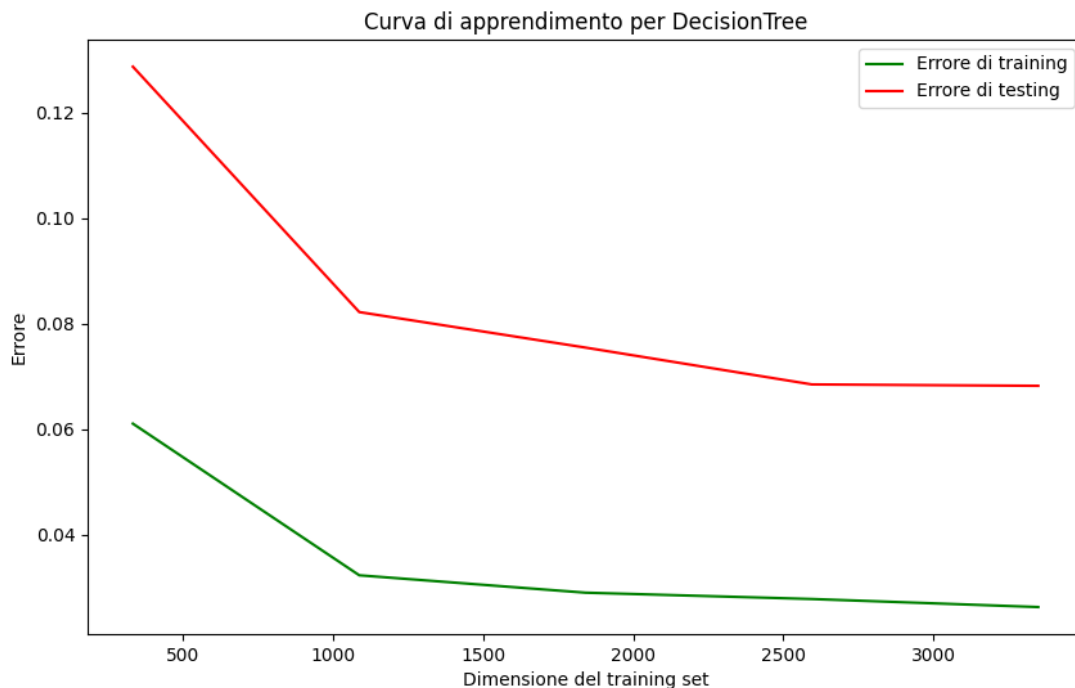
Le curve di apprendimento aiutano a capire:

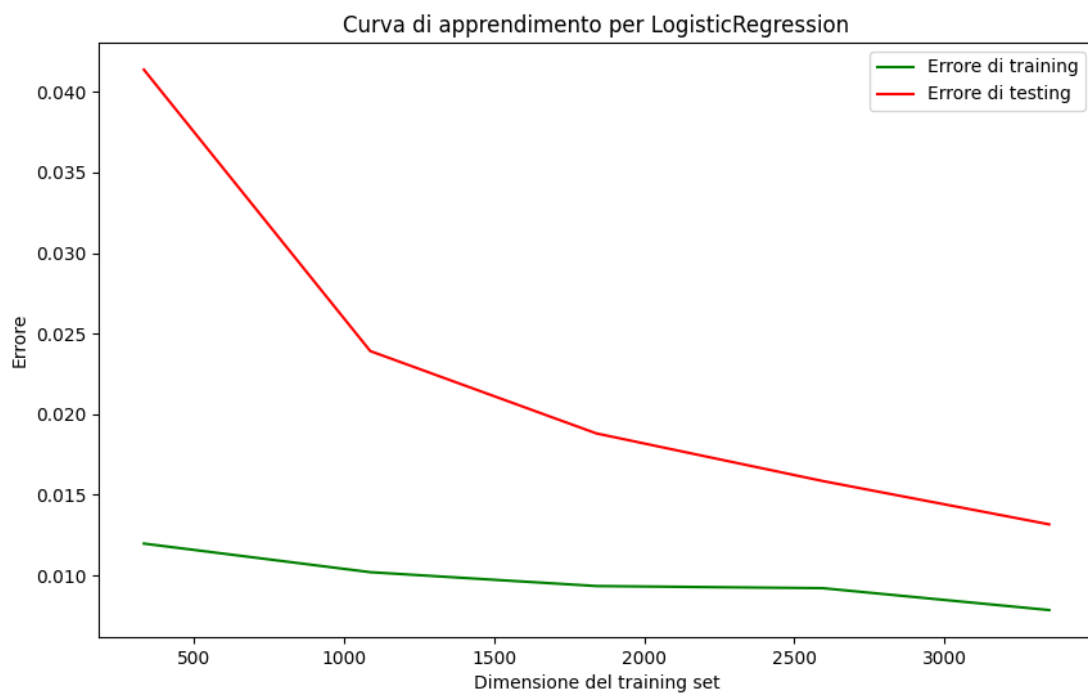
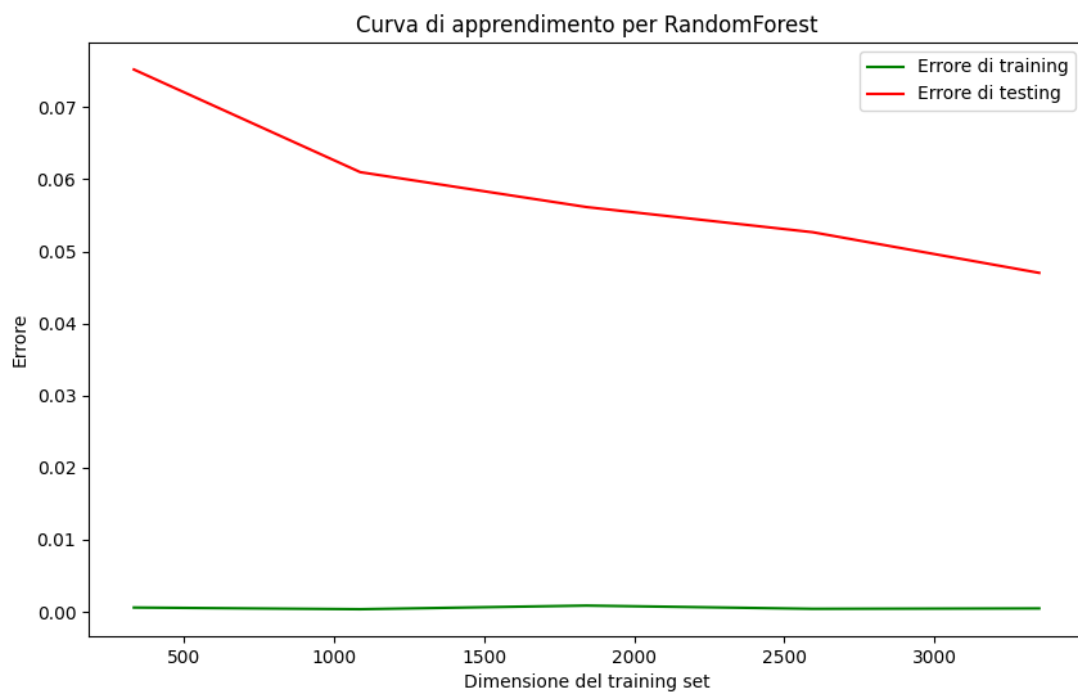
- Se il modello soffre di **overfitting** (errore di test molto più alto di quello di training).
- Se il modello soffre di **underfitting** (entrambi gli errori sono alti).

Nel mio caso si può denotare **overfitting**, il modello si adatta troppo ai dati di training, catturando anche il rumore invece della relazione generale.

Questo porta a un'alta **accuratezza sul training set** ma una **bassa generalizzazione** sui dati di test.

Fortunatamente i valori di errore sono minimi e diminuiscono al crescere della dimensione del training set.





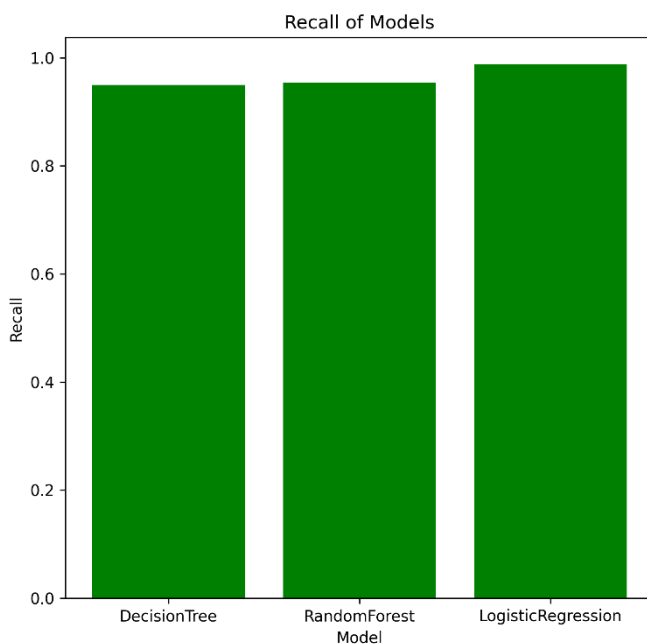
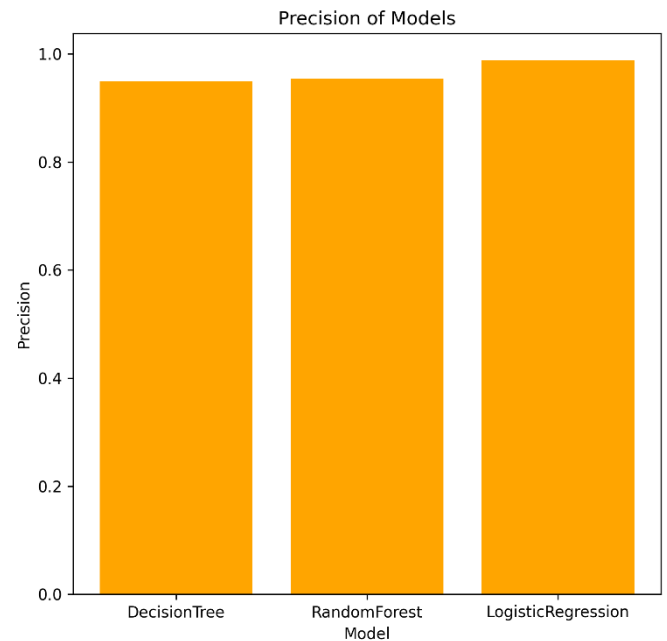
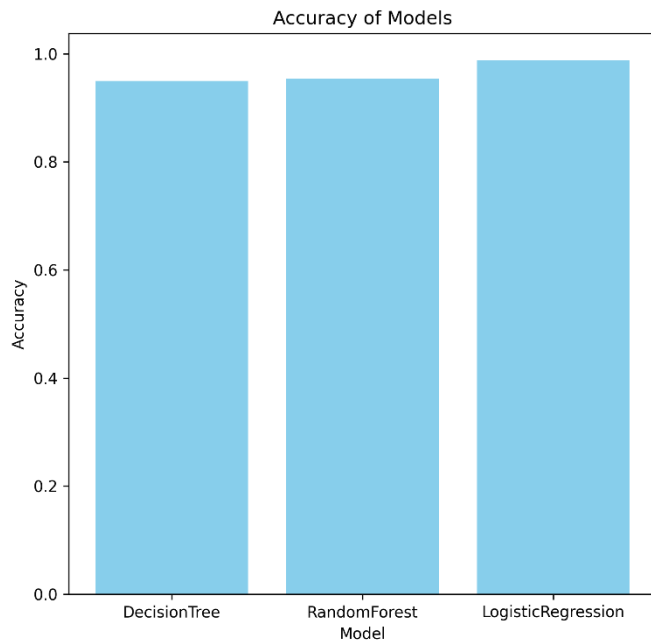
Fase 4) Valutazione delle prestazioni

La K-Fold Cross Validation, come già detto precedentemente, permette di addestrare il modello diverse volte utilizzando ogni volta esempi di training e di test diversi, in modo tale da ottenere delle valutazioni più “veritiere” rispetto alle reali performance del sistema.

Per la valutazione delle performance del sistema ho deciso di utilizzare le seguenti metriche:

- **Accuracy:** L'accuracy è una misura generale della correttezza del modello e rappresenta la frazione di previsioni corrette rispetto al totale delle previsioni.
- **Precision:** La precision macro è la media delle precisioni calcolate per ogni classe. La precisione per ogni classe è il numero di istanze di classe **c** classificate nella classe **c** / numero di istanze classificate nella classe **c**
- **Recall:** La recall macro è la media delle recall calcolate per ogni classe. La recall per ogni classe è il numero di istanze di classe **c** classificate nella classe **c** / numero di istanze nella classe **c**
- **F1:** La F1 macro è la media delle F1 calcolate per ogni classe. La F1 calcolata per ogni classe è la media armonica tra precision e recall (dunque è alta solo se entrambi i valori sono alti) ed è calcolata come segue: $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$

RISULTATI



Possiamo notare che in generale i valori sono molto buoni per ogni metrica in ogni modello. Questo potrebbe rappresentare un segnale di *overfitting*. Per verificare la presenza di *overfitting* bisogna prendere in considerazione altri fattori, come la varianza, la deviazione standard e le curve di

apprendimento.

Si verifica in presenza di sovra-adattamento ai dati di addestramento, imparando correlazioni falsate presenti nel dataset e non riuscendo a generalizzare bene.

Varianza e deviazione standard delle curve di apprendimento

Varianza : È la media dei quadrati delle differenze tra ciascun dato e la media.

Deviazione Standard : Radice quadrata della varianza

Nel contesto dell'apprendimento automatico misurano entrambe la dispersione dell'errore. Alti valori suggeriscono che il modello è sensibile alle variazioni dei dati in input, e dunque suggeriscono una possibile presenza di *overfitting*.

Modello	Varianza	Deviazione Standard
Decision Tree	0,00006724	0.0082
Random Forest	0,00007056	0.0084
Logistic Regression	0,00001849	0.0043

Analisi dei risultati per ogni modello

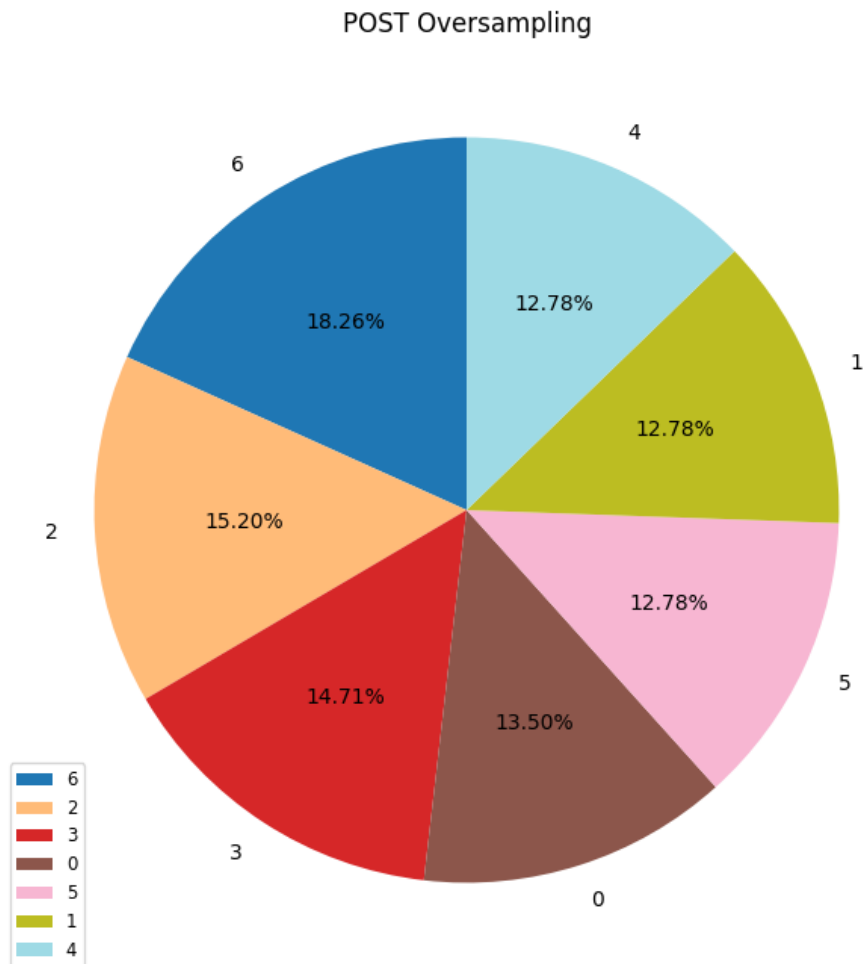
- **Decision Tree:** I valori di deviazione standard e varianza sono relativamente basse. Tuttavia, la varianza del test error è molto più alta rispetto a quello del train error, ciò potrebbe suggerire *overfitting*. Ciò potrebbe essere sintomo di *overfitting*. Tutto ciò suggerisce che il Decision Tree potrebbe soffrire di *overfitting*.
- **Random Forest:** In questo caso i valori di deviazione standard e varianza sono ancor più bassi, e in particolar modo la differenza tra le due è ancora più marcato rispetto al Decision Tree. La curva parte con un errore lievemente più basso che diminuisce molto rapidamente avvicinandosi a quello di training. Tuttavia è ancora evidente l'*overfitting*.
- **Logistic Regression:** I valori di varianza e deviazione sono molto bassi, ($>1/10$ rispetto gli altri modelli). Troviamo in generale un comportamento nelle curve che non suggerisce *overfitting*.

POSSIBILE SOLUZIONE

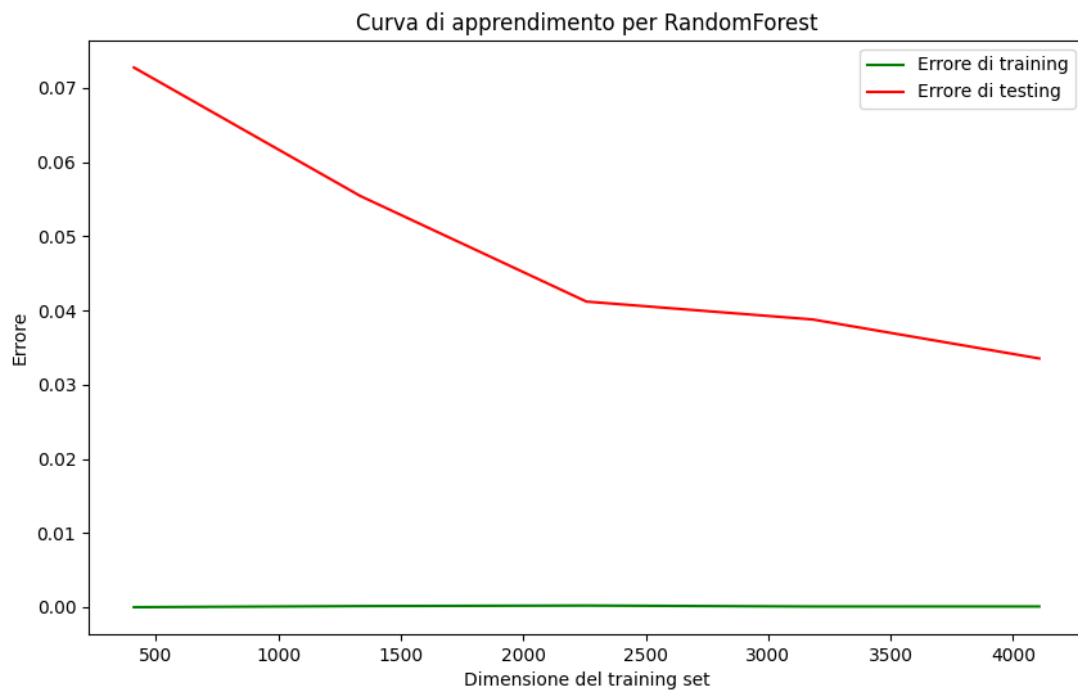
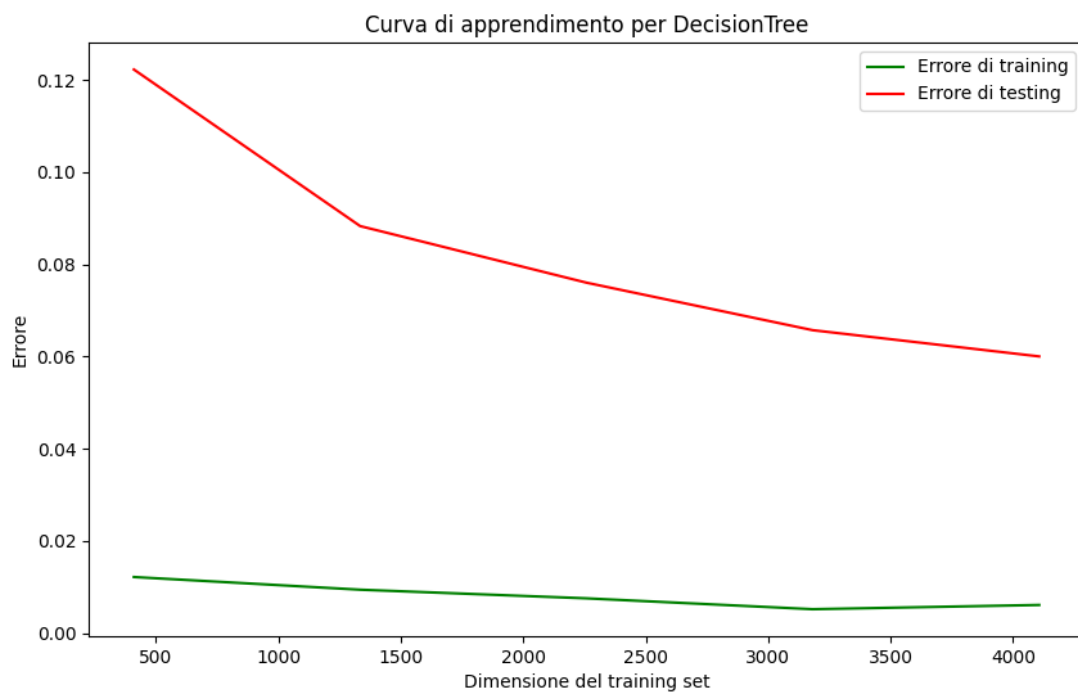
L'*overfitting* potrebbe dipendere da uno sbilanciamento delle classi. Una possibile soluzione è effettuare over-sampling. In questo caso ho individuato la classe con più esempi e ho generato esempi sintetici per le altre classi, facendo in modo che le classi fossero più equilibrate. Per fare ciò ho utilizzato una tecnica nota come *SMOTE*.

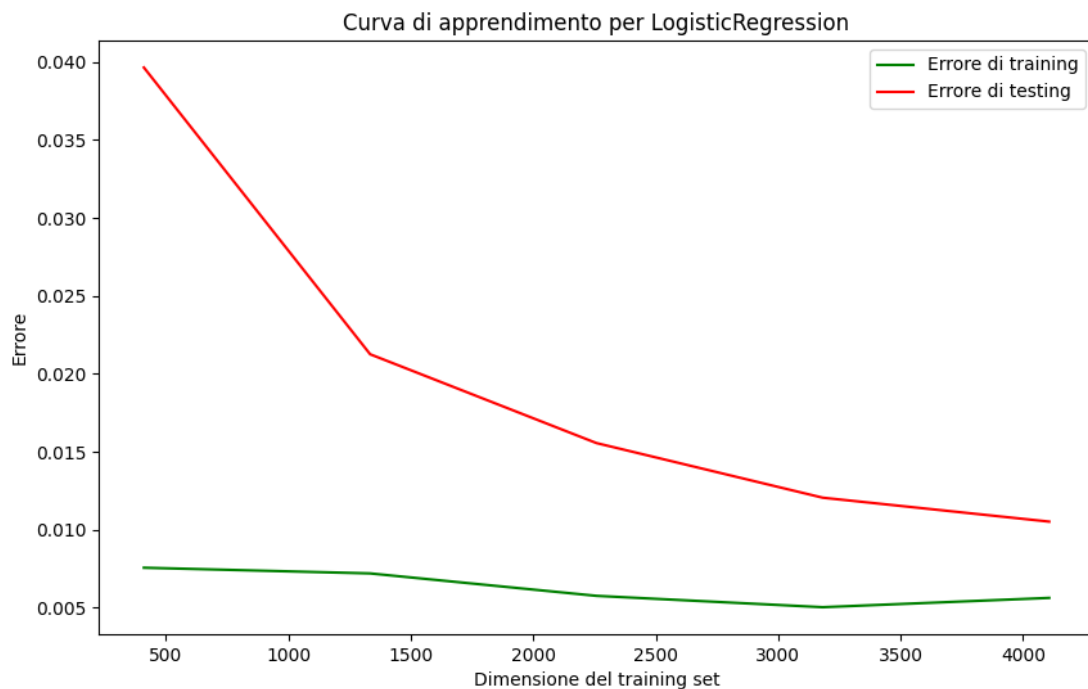
La tecnica *SMOTE* per ogni esempio nelle classi di minoranze identifica i suoi k vicini più prossimi e, mediante combinazione lineari tra questi ultimi e l'esempio stesso, genera nuovi dati sintetici.

Bilanciamento delle classi dopo l'*oversampling*



Abbiamo bilanciato le classi più deboli in modo tale da non stravolgere completamente il clustering eseguito in precedenza





In generale possiamo notare come per tutti i modelli i valori di deviazione standard e varianza si sono ridotti, le curve di apprendimento suggeriscono un miglioramento in tutti i casi.

Conclusione: L'*oversampling* sembra aver risolto l'*overfitting* e migliorato i diversi modelli anche se minimamente e si nota maggiormente con l'aumento della dimensione del training-set

Ragionamento probabilistico e Bayesian Network

Il ragionamento probabilistico è una forma di ragionamento che sfrutta la teoria della probabilità, in particolar modo dipendenza e indipendenza tra variabili e regola di Bayes. Nel ragionamento probabilistico si assegnano probabilità a ipotesi ed eventi e si utilizzano le probabilità a posteriori per i ragionamenti. Un'applicazione del ragionamento probabilistico sono le reti bayesiane.

Queste vengono rappresentate mediante grafi orientati aciclici (DAG) dove ogni nodo del grafo rappresenta una variabile e gli archi indicano le dipendenze probabilistiche tra le variabili.

Per questa parte di progetto sono state utilizzate le librerie *networkx* e *matplotlib* per la visualizzazione di grafo e grafici e *pgmpy* per gestire la rete bayesiana.

Valori continui

STRUTTURA RETE BAYESIANA

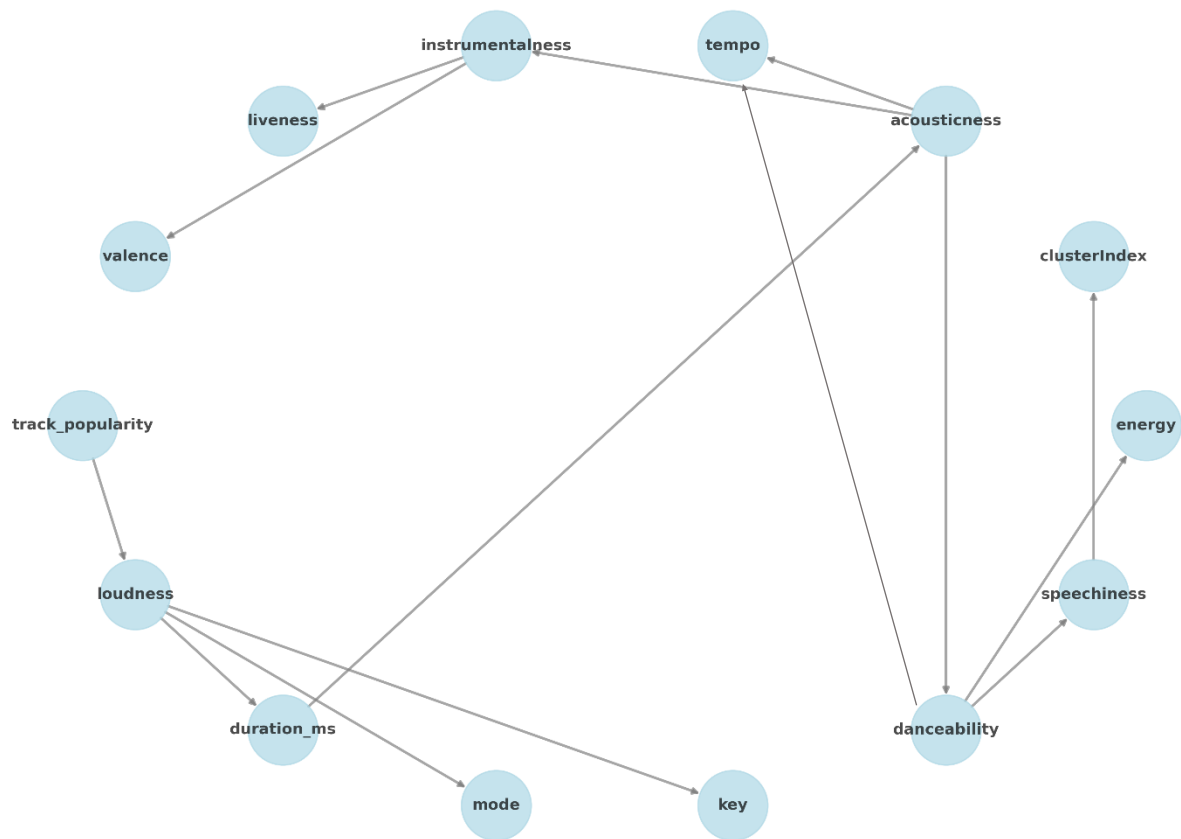
Per poter utilizzare una rete bayesiana, è necessario identificare le variabili e le dipendenze tra esse. Nel mio caso, le variabili sono le feature delle canzoni e l'indice del cluster. Il problema principale è quindi l'apprendimento della struttura ottimale della rete.

Inizialmente, ho utilizzato *HillClimbSearch*, un algoritmo di ricerca locale che esplora lo spazio delle possibili strutture per massimizzare la funzione di verosimiglianza del dataset, valutata tramite il *K2 Score*. Tuttavia, ho riscontrato diverse difficoltà:

- Quando il numero di iterazioni dell'algoritmo era maggiore o uguale a 4, il consumo di memoria cresceva rapidamente fino a decine di GiB o addirittura TiB, portando al crash del sistema.
- Anche eseguendo il processo su piattaforme cloud come Google Colab, le risorse computazionali non erano sufficienti.

Per superare questo problema, ho optato per una struttura arbitraria, in cui tutte le feature di input dipendono direttamente dalla feature di output. Per determinare le dipendenze tra le altre variabili, ho sfruttato la documentazione ufficiale di Spotify sulle metriche delle tracce, cercando di individuare correlazioni significative tra le feature.

Rete Bayesiana



Come possiamo vedere *clusterIndex* è una variabile indipendente, dunque il suo valore non dipende dalle altre variabili.

Danceability, invece, dipende da variabili quali *clusterIndex*, *energy* e *tempo*.

La rete bayesiana permette di effettuare un semplice task di classificazione e molto altro:

- **Generazione di sample:** Vengono sfruttate le probabilità apprese dal dataset in fase di addestramento per generare un esempio sintetico credibile
- **Gestione di dati mancanti:** Le reti bayesiane sono in grado di gestire casi in cui per una variabile casuale non sia stato osservato alcun valore

Problema delle reti bayesiane con valori continui

Le reti Bayesiane, rispetto agli altri modelli di apprendimento, presentano un problema nel caso in cui i valori delle feature sono continui e non discreti. Infatti, per creare le *CPD*, è richiesta l'osservazione di ogni singolo valore. Se ciò non avviene la rete bayesiana restituisce degli errori.

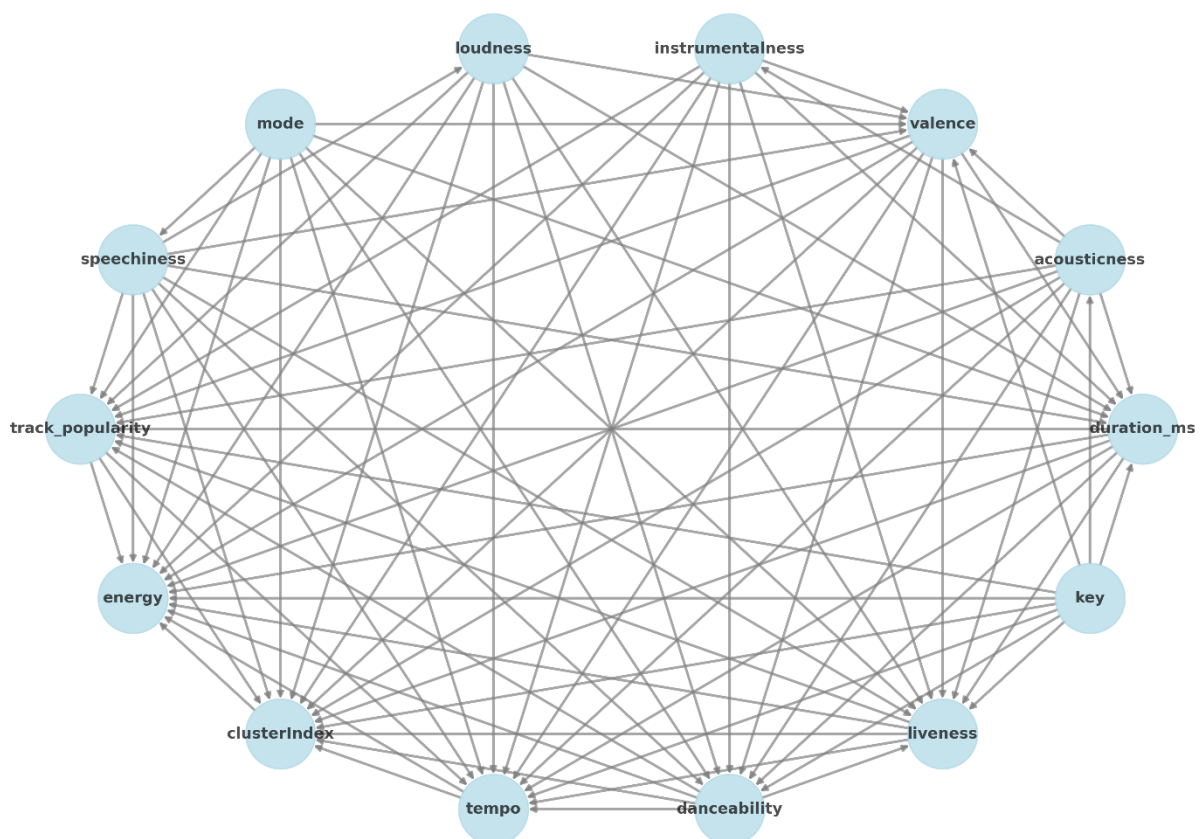
In questo caso ho provato a predire il cluster per questa canzone che non appartiene al mio dataset e il programma ha generato un errore in quanto uno dei valori delle metriche non era mai stato osservato fino a quel momento. Questo è un problema tipico delle reti bayesiane, in quanto per costruire le *CPD* le reti necessitano di aver osservato tutti i valori possibili (nel mio caso per ogni metrica tutti i possibili valori compresi tra 0 e 1 generati dal *MinMaxScaler*).

Valori discreti

Per risolvere questo problema ho deciso di discretizzare i valori continui mediante *KBinsDiscretizer* (presente nella libreria *scikit_learn*) il quale permette di trasformare i valori continui in discreti.

L'utilizzo dei valori discreti, oltre a risolvere i problemi dei valori continui, mi ha permesso anche di risolvere i problemi relativamente all'utilizzo della memoria, che prima riscontravo.

Adesso è stato reso possibile l'utilizzo del metodo *HillClimbSearch* per apprendere la struttura della rete. Possiamo vedere come la struttura è molto più complessa rispetto a quella da me ipotizzata.



Adesso la rete è riuscita a predire il cluster su canzoni non appartenenti al dataset. Ovviamente l'esempio randomico viene generato già discretizzato.

Ragionamento logico e Prolog

Il ragionamento logico si differisce da quello probabilistico in quanto in questo caso il ragionamento avviene usando la logica matematica. Viene costruita una cosiddetta *Knowledge Base*, contenente gli assiomi (ovvero affermazioni sempre vere). Gli assiomi si dividono a loro volta in regole e fatti. I fatti sono verità immutabili, una regola è una dichiarazione che afferma che qualcosa è vero in base ad altre cose che sono vere.

Ho utilizzato la libreria *pyswip* per il *Prolog*.

Prolog è un linguaggio di programmazione dichiarativo basato sul ragionamento logico. Nel mio caso sono stati definiti 2 tipi di fatti e 1 regola

- Fatto song: contiene le informazioni relative a una canzone
- Fatto clustered_song: contiene le informazioni relative a una canzone e al suo cluster di appartenenza
- Regola: canzoni_info(NomeCanzone, Autore, Cluster) :-
 - o song(A1, B1, C1, D1, E1, F1, G1, H1, I1, L1, Autore, NomeCanzone)
 - o clustered_song(A2, B2, C2, D2, E2, F2, G2, H2, I2, L2, Cluster),

Questa regola può essere utilizzata per diversi scopi. Uno tra tanti consiste nel ricavare il cluster in cui ogni canzone è stata inserita.

O ancora, come ho implementato, effettuare delle query per interrogare la KB e ricevere risultati inerenti canzoni che rispettino i criteri stabiliti.

Questo è una piccola parte dell'output restituito.

```
Scegli una query:
1. Cerca canzoni per durata
2. Cerca canzoni per popolarità
3. Esci
▶ Inserisci il numero della query: 1
▶ Inserisci la durata della canzone (1. breve -b, 2. media -m, 3. lunga -l): 1

✅ Canzoni trovate per durata '1':
🎵 Are You with Me - Dash Berlin Radio Edit (duration: 167975 ms)
🎵 Love & War (duration: 178693 ms)
🎵 Did I Mention (duration: 154999 ms)
🎵 Lightning Bolt (duration: 142747 ms)
...

Scegli una query:
1. Cerca canzoni per durata
2. Cerca canzoni per popolarità
3. Esci
▶ Inserisci il numero della query: 1
▶ Inserisci la durata della canzone (1. breve -b, 2. media -m, 3. lunga -l): 3

✅ Canzoni trovate per durata '3':
🎵 I'm Getting Ready (duration: 506200 ms)
🎵 Sultans Of Swing (duration: 350261 ms)
🎵 The Zone (duration: 418507 ms)
🎵 Deuces Remix (duration: 404213 ms)
...
```

Potrebbe essere usata anche per altri scopi (esempio: dato il nome di una canzone vedere chi è l'autore e capire in quale cluster è stata inserita).

In foto sono mostrate solo 2 query con 3 criteri possibili di specifiche per un puro scopo dimostrativo e non effettivo, ma si potrebbero inserire query per qualsiasi criterio/features e anche in qualsiasi combinazione.

Sviluppi futuri

Il progetto da un punto di vista funzionale potrebbe interfacciarsi direttamente a Spotify e magari diventarne una funzionalità aggiuntiva per completarne le funzionalità o semplicemente per creare, sul profilo dell'utente, le nuove playlist.

Si potrebbe dare la possibilità all'utente di interfacciarsi con il sistema in modo tale da classificare e inserire automaticamente le canzoni nelle nuove playlist. Il tutto potrebbe essere eseguito con un'interfaccia grafica.

Da un punto di vista progettuale si potrebbe applicare la *PCA* o altre tecniche di *feature selection* per provare a ridurre la dimensionalità del problema.

Riferimenti bibliografici

- **Ragionamento logico:** D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. 3/e. Cambridge University Press [Ch.5]
- **Apprendimento supervisionato:** D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. Cambridge University Press. 3rd Ed. [Ch.7]
- **Ragionamento probabilistico e reti bayesiane:** D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. Cambridge University Press. 3/e. [Ch.9]
- **Apprendimento non supervisionato:** D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. Cambridge University Press. 3rd Ed. [Ch.10]
- **Prolog:** D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. 3e, Cambridge University Press [Ch.15]